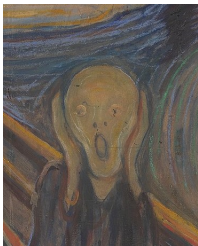


Les pointeurs

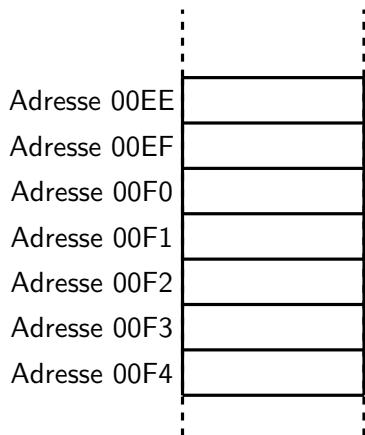
B.U.T. Informatique



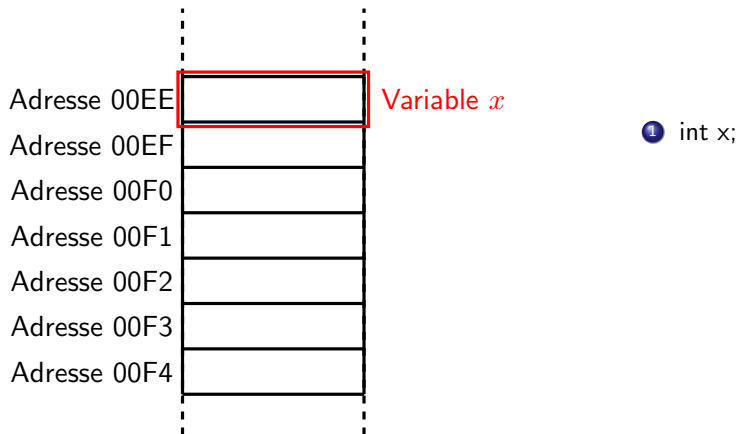
Objectifs d'apprentissage

- Connaître les pointeurs
- Comprendre comment fonctionne la mémoire avec les variables et les pointeurs
- Manipuler les pointeurs
- Tableaux dynamiques

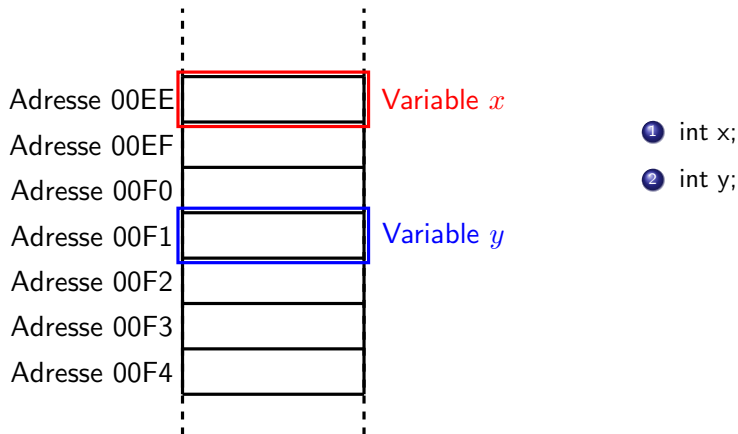
Exemple



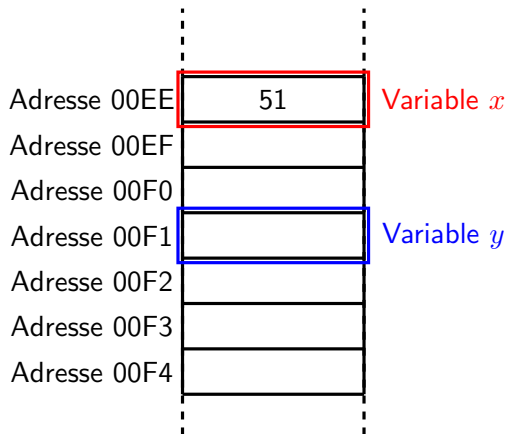
Exemple



Exemple

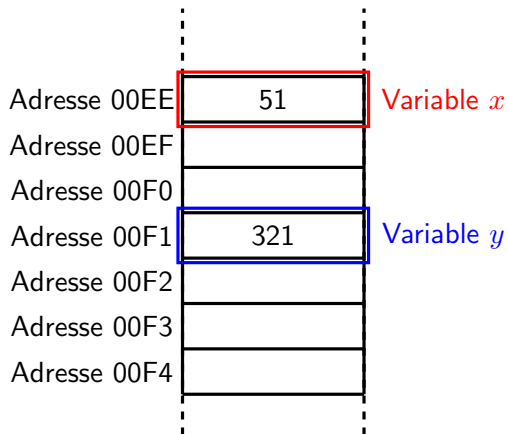


Exemple



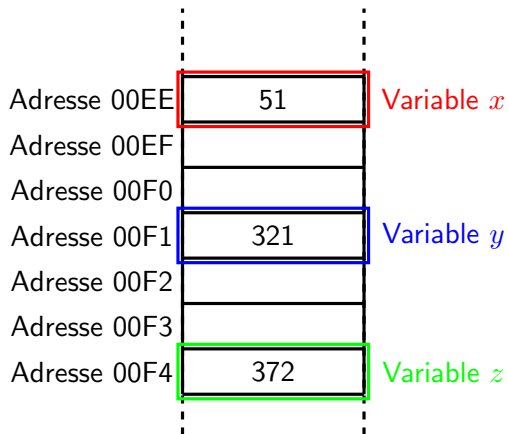
- ① `int x;`
- ② `int y;`
- ③ `x = 51;`

Exemple



- ① `int x;`
- ② `int y;`
- ③ `x = 51;`
- ④ `y = 321;`

Exemple



- ① `int x;`
- ② `int y;`
- ③ `x = 51;`
- ④ `y = 321;`
- ⑤ `int z = x + y;`

Une variable

Une variable, au final, c'est une adresse qui contient une valeur.

- *var1* c'est la **valeur** de la variable *var1*
- *&var* c'est l'**adresse** de la variable *var1*

Dans le diapo précédent, *&x* est égal à 00EE et *&y* est égal à 00F1

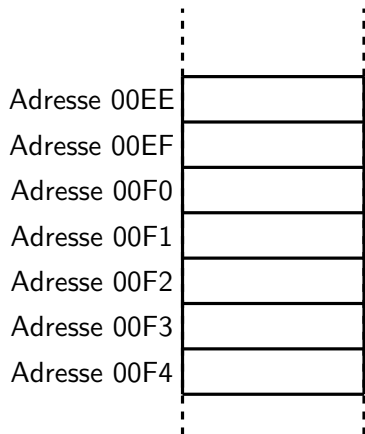
Les pointeurs

Un pointeur

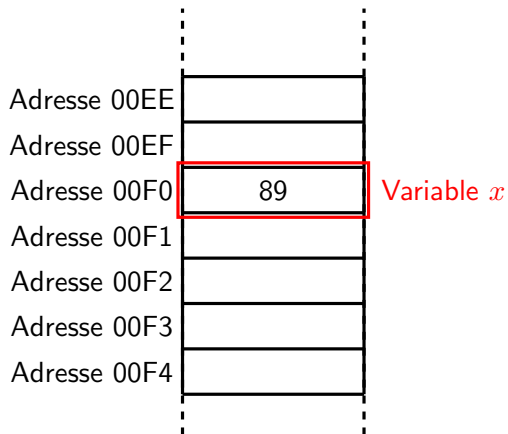
Un pointeur est un objet dont la valeur est égale à l'adresse d'une variable. Le type du pointeur dépend de la variable sur lequel il pointe.

- Tout les langages n'ont pas forcément la possibilité d'utiliser des pointeurs
- Un pointeur c'est une variable qui stocke une adresse
- Note : une adresse c'est un entier
- Un pointeur qui ne pointe sur aucune adresse, pointe sur **NULL**
- * permet d'accéder à la valeur pointée
- C++ : `TYPE *NOMPOINTEUR;`

Exemple

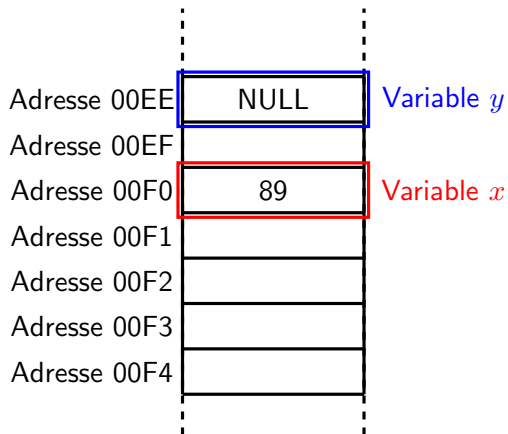


Exemple



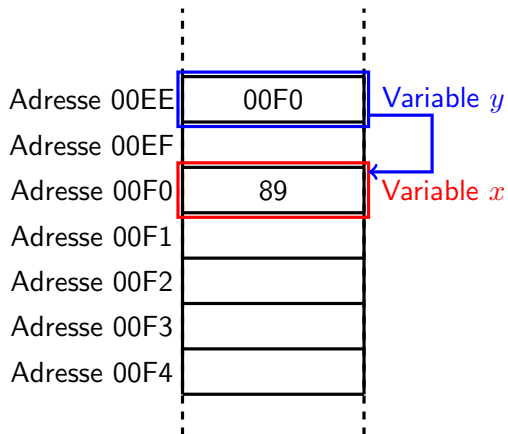
① `int x = 89;`

Exemple



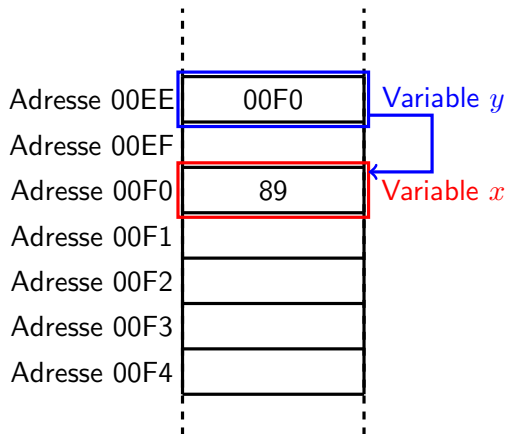
- ① `int x = 89;`
- ② `int *y = NULL;`

Exemple



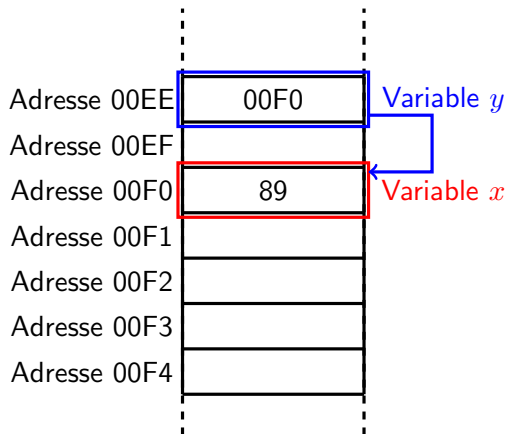
- 1 `int x = 89;`
- 2 `int *y = NULL;`
- 3 `y = &x;`

Exemple



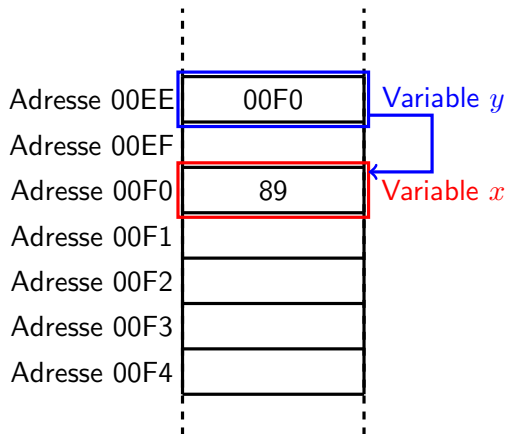
- ① `int x = 89;`
- ② `int *y = NULL;`
- ③ `y = &x;`
- ④ `Afficher(&y);`

Exemple



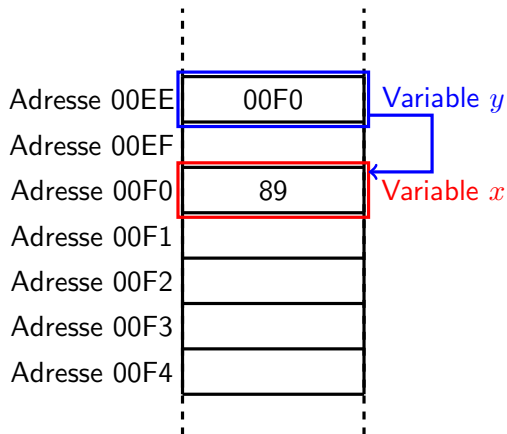
- 1 `int x = 89;`
- 2 `int *y = NULL;`
- 3 `y = &x;`
- 4 `Afficher(&y); // 00EE`

Exemple



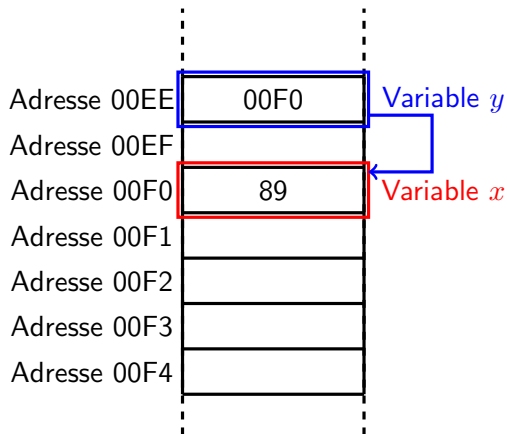
- 1 `int x = 89;`
- 2 `int *y = NULL;`
- 3 `y = &x;`
- 4 `Afficher(&y); // 00EE`
- 5 `Afficher(y);`

Exemple



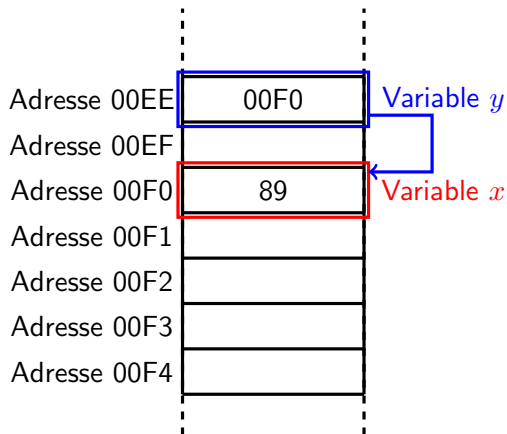
- ① `int x = 89;`
- ② `int *y = NULL;`
- ③ `y = &x;`
- ④ `Afficher(&y); // 00EE`
- ⑤ `Afficher(y); // 00F0`

Exemple



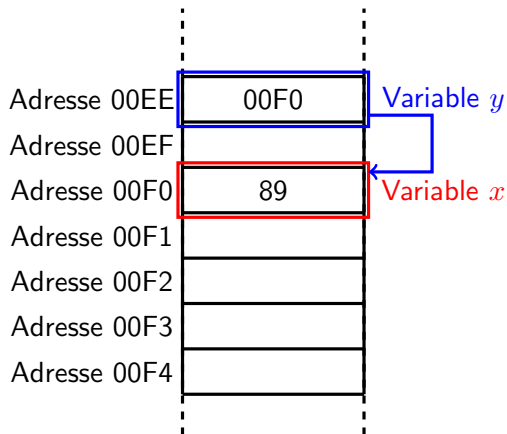
- ① `int x = 89;`
- ② `int *y = NULL;`
- ③ `y = &x;`
- ④ `Afficher(&y); // 00EE`
- ⑤ `Afficher(y); // 00F0`
- ⑥ `Afficher(*y);`

Exemple



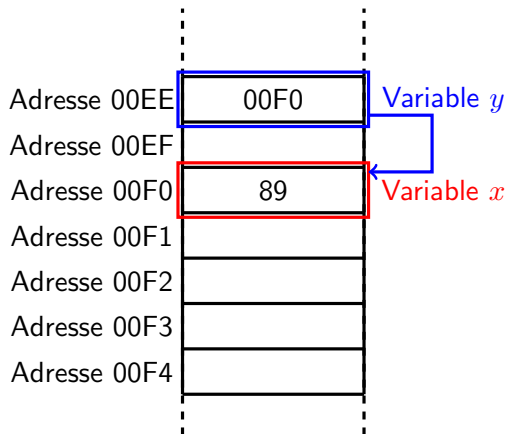
- ① `int x = 89;`
- ② `int *y = NULL;`
- ③ `y = &x;`
- ④ `Afficher(&y); // 00EE`
- ⑤ `Afficher(y); // 00F0`
- ⑥ `Afficher(*y); // 89`

Exemple



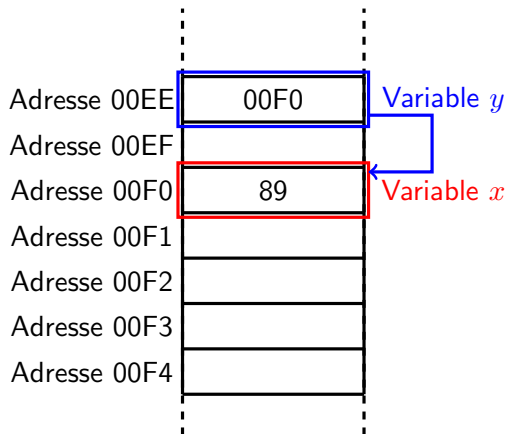
- 1 `int x = 89;`
- 2 `int *y = NULL;`
- 3 `y = &x;`
- 4 `Afficher(&y); // 00EE`
- 5 `Afficher(y); // 00F0`
- 6 `Afficher(*y); // 89`
- 7 `Afficher(x);`

Exemple



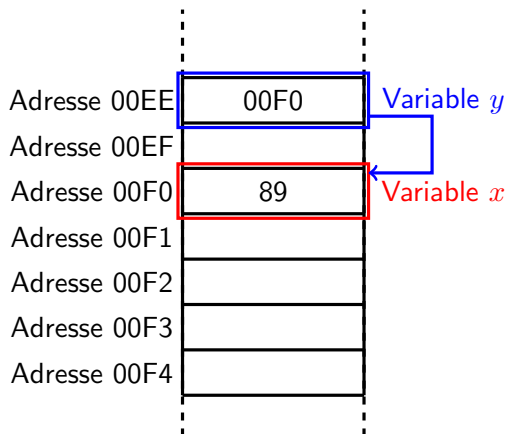
- ① `int x = 89;`
- ② `int *y = NULL;`
- ③ `y = &x;`
- ④ `Afficher(&y); // 00EE`
- ⑤ `Afficher(y); // 00F0`
- ⑥ `Afficher(*y); // 89`
- ⑦ `Afficher(x); // 89`

Exemple



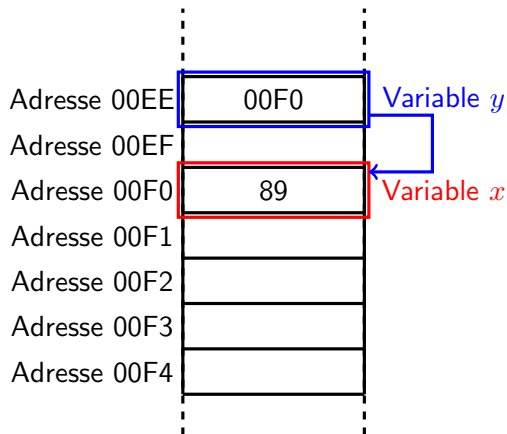
- ① `int x = 89;`
- ② `int *y = NULL;`
- ③ `y = &x;`
- ④ `Afficher(&y); // 00EE`
- ⑤ `Afficher(y); // 00F0`
- ⑥ `Afficher(*y); // 89`
- ⑦ `Afficher(x); // 89`
- ⑧ `Afficher(&x);`

Exemple

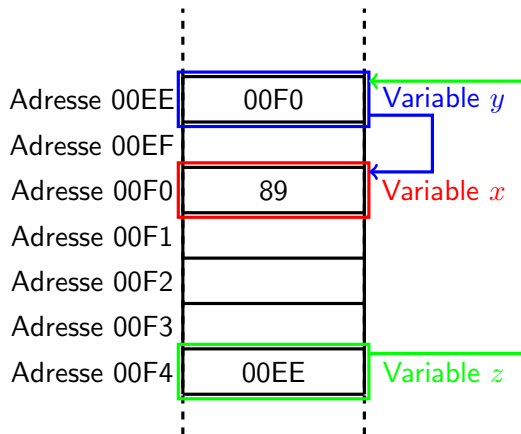


- ❶ `int x = 89;`
- ❷ `int *y = NULL;`
- ❸ `y = &x;`
- ❹ `Afficher(&y); // 00EE`
- ❺ `Afficher(y); // 00F0`
- ❻ `Afficher(*y); // 89`
- ❼ `Afficher(x); // 89`
- ❽ `Afficher(&x); // 00F0`

Exemple

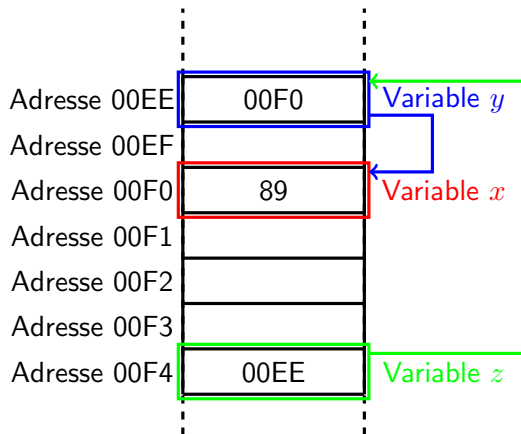


Exemple



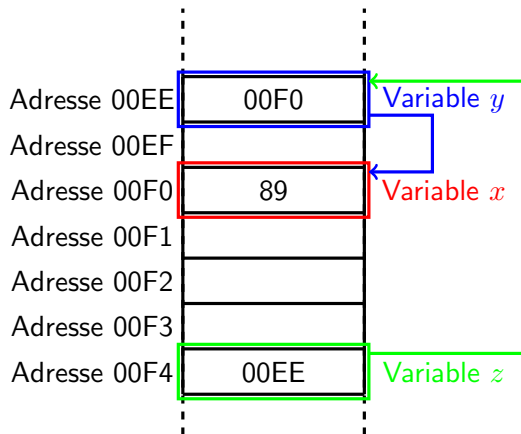
1 `int **z = &y;`

Exemple



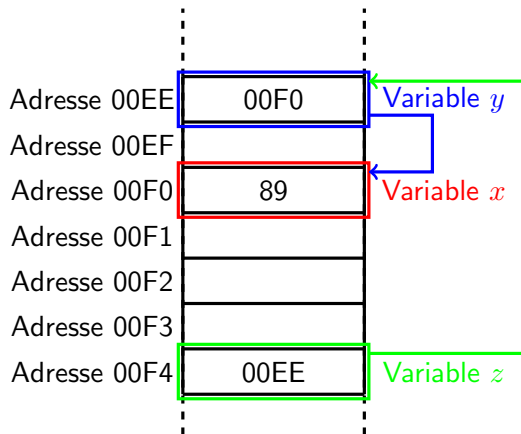
- 1 `int **z = &y;`
- 2 `Afficher(z);`

Exemple



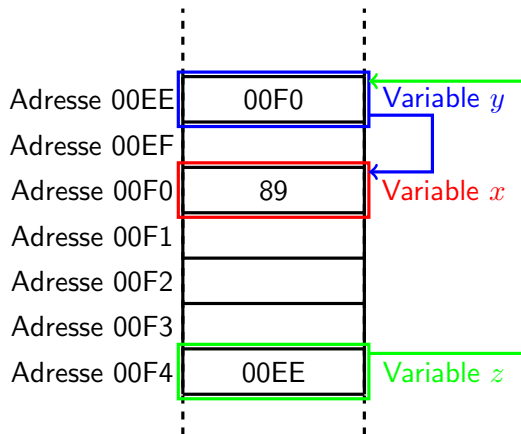
- 1 `int **z = &y;`
- 2 `Afficher(z); // 00EE`

Exemple



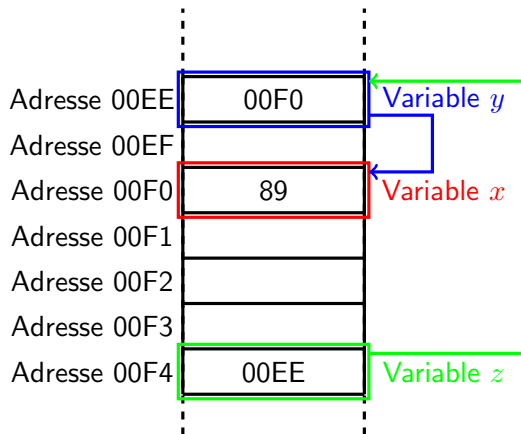
- 1 `int **z = &y;`
- 2 `Afficher(z); // 00EE`
- 3 `Afficher(&z);`

Exemple



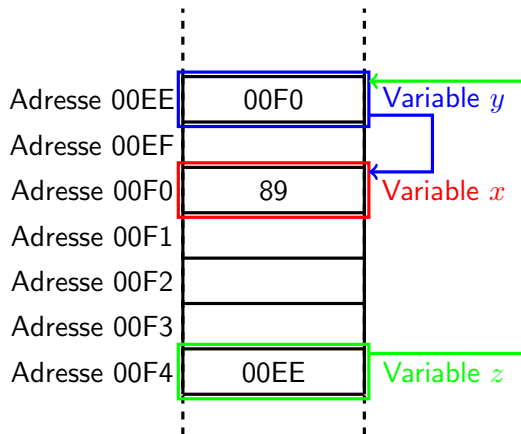
- 1 `int **z = &y;`
- 2 `Afficher(z); // 00EE`
- 3 `Afficher(&z); // 00F4`

Exemple



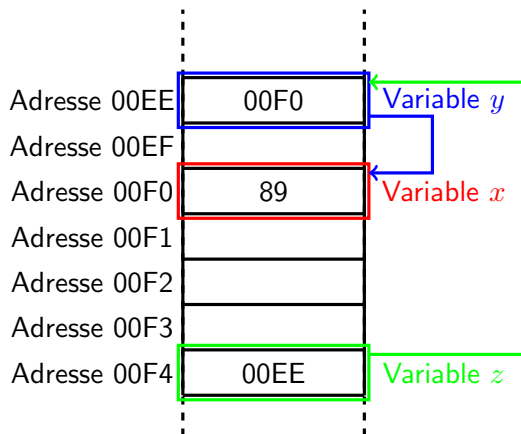
- 1 `int **z = &y;`
- 2 `Afficher(z); // 00EE`
- 3 `Afficher(&z); // 00F4`
- 4 `Afficher(*z);`

Exemple



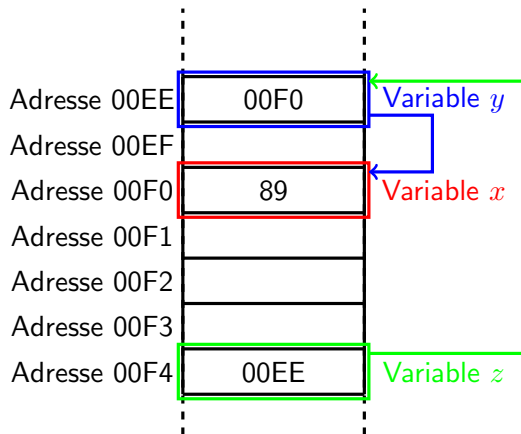
- ❶ `int **z = &y;`
- ❷ `Afficher(z); // 00EE`
- ❸ `Afficher(&z); // 00F4`
- ❹ `Afficher(*z); // 00F0`

Exemple



- ❶ `int **z = &y;`
- ❷ `Afficher(z); // 00EE`
- ❸ `Afficher(&z); // 00F4`
- ❹ `Afficher(*z); // 00F0`
- ❺ `Afficher(**z);`

Exemple



- ❶ `int **z = &y;`
- ❷ `Afficher(z); // 00EE`
- ❸ `Afficher(&z); // 00F4`
- ❹ `Afficher(*z); // 00F0`
- ❺ `Afficher(**z); // 89`



Allocation dynamique

Jusqu'à maintenant, le compilateur gère l'**allocation** et la **libération** de la mémoire

- Allocation statique : les espaces mémoires sont réservé au lancement du programme (pour les langages avec compilateurs)
- Libération : la mémoire occupée est libérée à la fin du programme



Allocation dynamique

Jusqu'à maintenant, le compilateur gère l'**allocation** et la **libération** de la mémoire

- Allocation statique : les espaces mémoires sont réservé au lancement du programme (pour les langages avec compilateurs)
- Libération : la mémoire occupée est libérée à la fin du programme

On va maintenant apprendre à gérer nous même la mémoire, c'est l'allocation dynamique

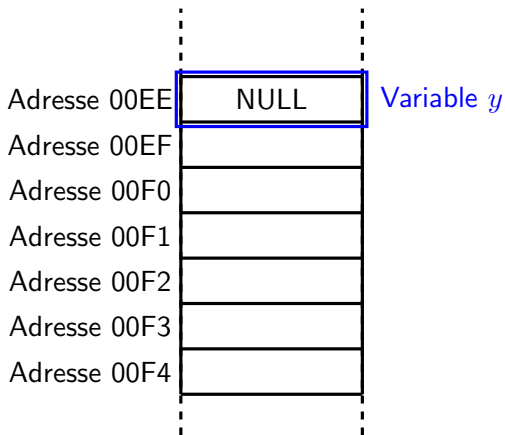
Allocation dynamique

Allocation dynamique

C'est réserver pendant l'exécution du programme l'espace mémoire nécessaire au fonctionnement de l'algorithme

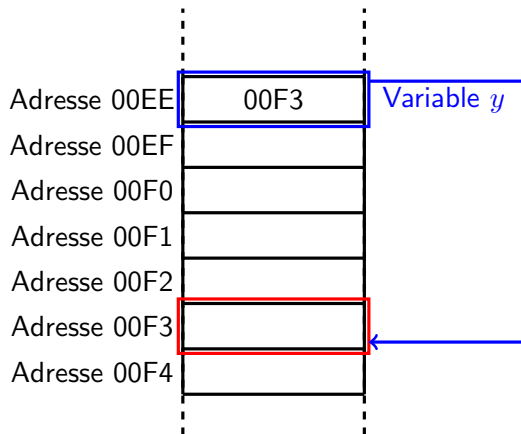
- Allocation en C++ : `POINTEUR = new TYPE;`
- Libération en C++ : `delete POINTEUR;`
- Attention aux fuites mémoires ! (Exemple après)

Exemple



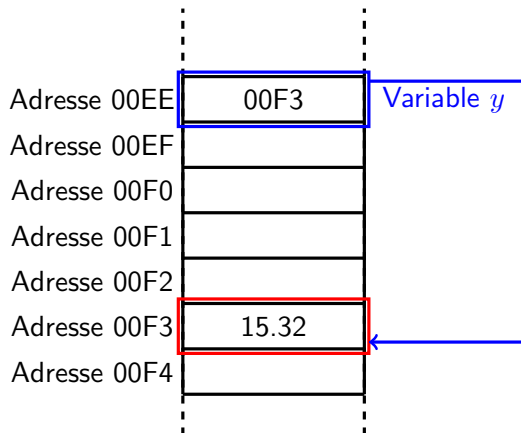
❶ `float *y = NULL;`

Exemple



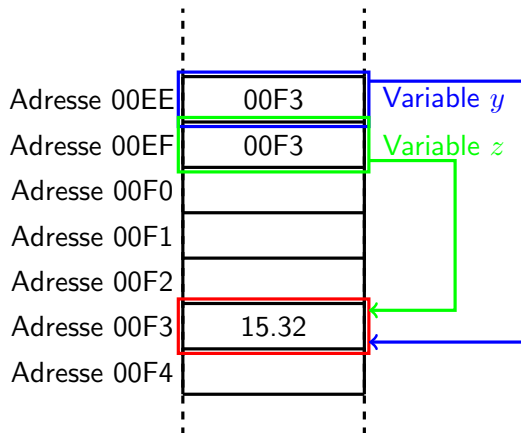
- ❶ `float *y = NULL;`
- ❷ `y = new float;`

Exemple



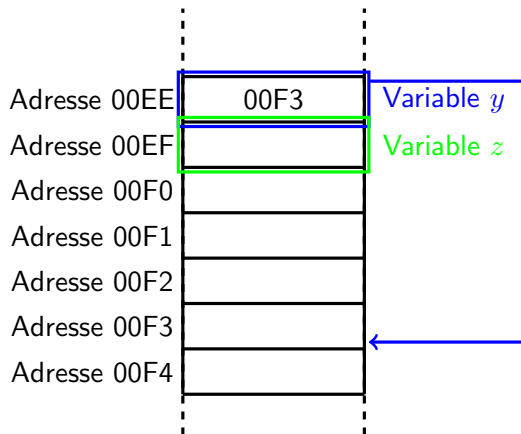
- ❶ `float *y = NULL;`
- ❷ `y = new float;`
- ❸ `*y = 15.32;`

Exemple



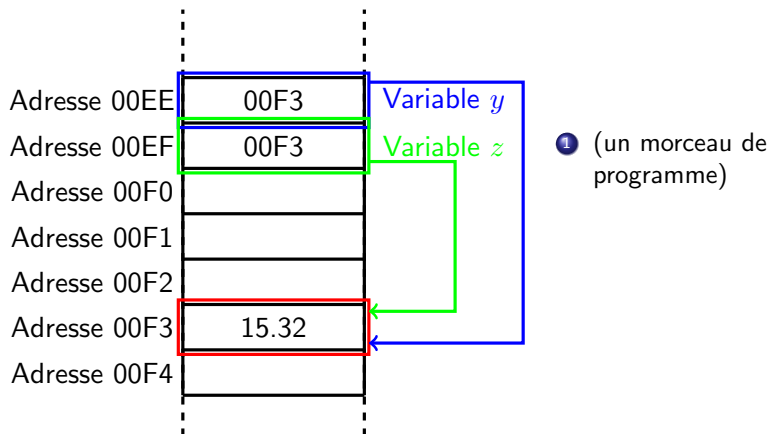
- 1 float *y = NULL;
- 2 y = new float;
- 3 *y = 15.32;
- 4 float *z = y;

Exemple

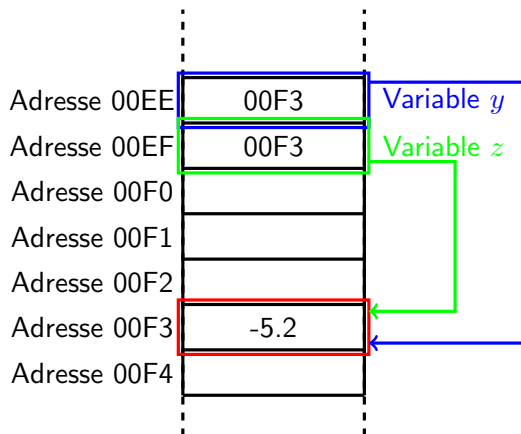


- 1 float $*y = \text{NULL};$
- 2 $y = \text{new float};$
- 3 $*y = 15.32;$
- 4 float $*z = y;$
- 5 delete $z;$

J'ai perdu la mémoire



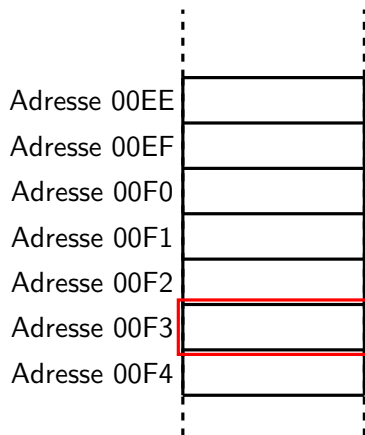
J'ai perdu la mémoire



① (un morceau de programme)

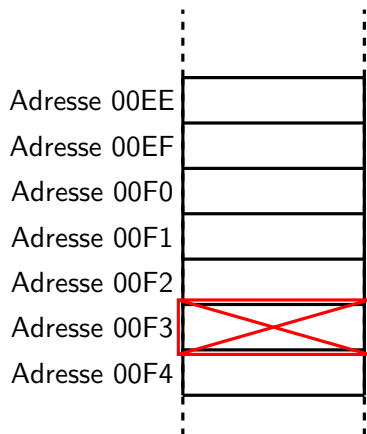
② $*y = -5.2$

J'ai perdu la mémoire



- ① (un morceau de programme)
- ② $*y = -5.2$
- ③ fin du programme

J'ai perdu la mémoire



- ❶ (un morceau de programme)
- ❷ $*y = -5.2$
- ❸ fin du programme
- ❹ Perte de mémoire

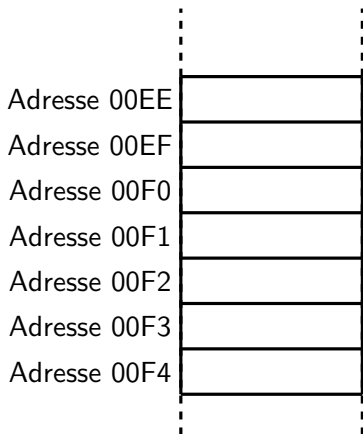
Retour sur les tableaux statique

- En réalité un pointeur est un entier (l'adresse)
- $\text{pointeur} + 1 \rightarrow$ c'est l'adresse suivante celle pointée
- `int tab[100]` $\rightarrow$ déclare un tableau tab de 100 entiers
- En réalité, tab est un pointeur vers le premier élément du tableau
- `tab[n]` $\rightarrow$ `*(tab+n)`

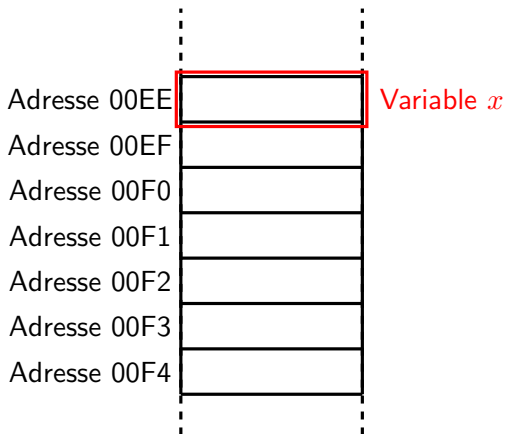
Retour sur les tableaux dynamique

- ❶ `int *y;`
- ❷ `int z;`
- ❸ `std::cin >> z;`
- ❹ `y = new int[z];` // Alloue l'espace de mémoire pour le tableau
- ❺ `y[0] = 455;`
- ❻ `y[1] = 55;`
- ❼ `*(y+2) = 5;` // Modifie la troisième case du tableau
- ❽ `delete [] y;` // Supprime toutes les cases du tableau

Exemple

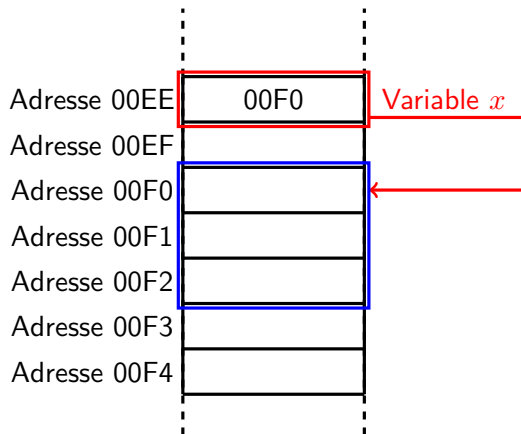


Exemple



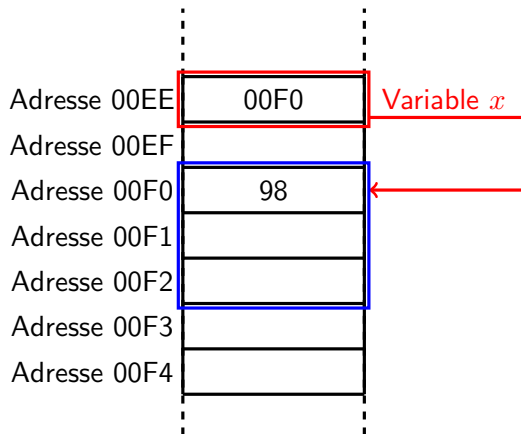
1 `int *x;`

Exemple



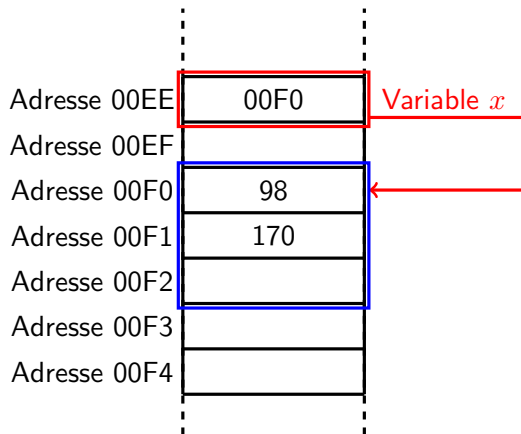
- 1 `int *x;`
- 2 `x = new int[3];`

Exemple



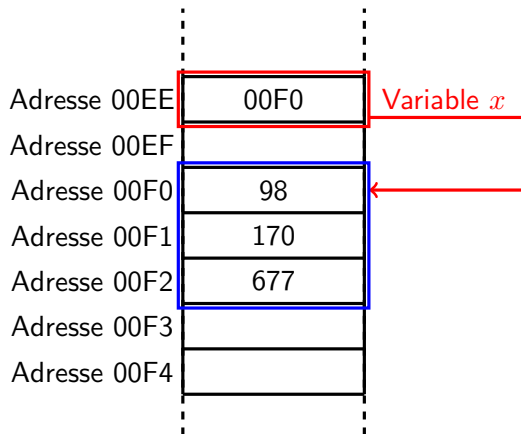
- 1 `int *x;`
- 2 `x = new int[3];`
- 3 `*x = 98;`

Exemple



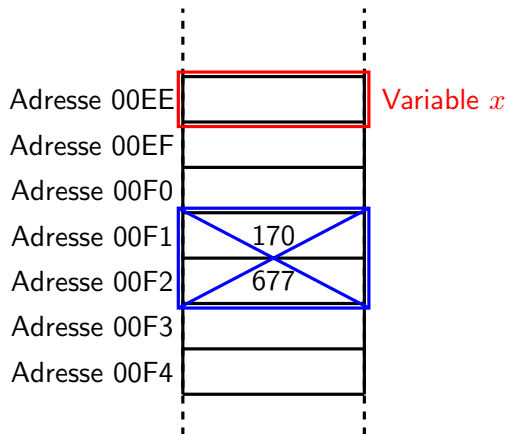
- 1 `int *x;`
- 2 `x = new int[3];`
- 3 `*x = 98;`
- 4 `*(x+1) = 170;`

Exemple



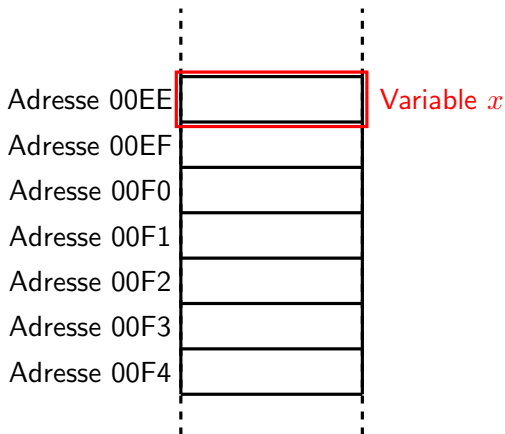
- 1 `int *x;`
- 2 `x = new int[3];`
- 3 `*x = 98;`
- 4 `*(x+1) = 170;`
- 5 `*(x+2) = 677;`

Exemple



- 1 `int *x;`
- 2 `x = new int[3];`
- 3 `*x = 98;`
- 4 `*(x+1) = 170;`
- 5 `*(x+2) = 677;`
- 6 `delete x;`

Exemple



- ① `int *x;`
- ② `x = new int[3];`
- ③ `*x = 98;`
- ④ `*(x+1) = 170;`
- ⑤ `*(x+2) = 677;`
- ⑦ `delete [] x;`